

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ ДЕРЖАВНИЙ
УНІВЕРСИТЕТ ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО
ЗАБЕЗПЕЧЕННЯ

Курсова робота

з дисципліни "Конструювання програмного забезпечення Java"
на тему: Інструмент управління та генерації паролів

Виконав: студент групи ПД-34 Сич Б.О

Перевірів: доц. Довженко Т.П.

Київ, 2025р.

ЗМІСТ

ВСТУП.....	3
1 АНАЛІЗ ПРОБЛЕМИ ТА ВИБІР ТЕХНОЛОГІЙ	5
1.1 Проблема генерації безпечних паролів	5
1.2 Аналіз існуючих рішень.....	7
1.3 Обґрунтування вибору технологій.....	9
2 ПРОЕКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ	12
2.1 Функціональні вимоги	12
2.2 Нефункціональні вимоги	15
2.3 Проектування БД.....	17
2.4 User Stories та діаграма послідовності.....	18
2.5 Алгоритми генерації паролів.....	21
2.5.1 Random.....	21
2.5.2 Entropy-based.....	22
2.5.3 Passphrase.....	22
3 РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	23
3.1 Структура проекту та опис модулів системи.....	23
3.2 Інтерфейс користувача	25
3.3 REST API та підключення Swagger UI	27
3.4 Тестування системи.....	29
ВИСНОВКИ	33
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	34
ДОДАТОК А	36
ДОДАТОК Б.....	38

ВСТУП

Зі стрімким розвитком інформаційних технологій та зростанням обсягів зберіганих в електронній формі даних зростає і потреба у надійних засобах захисту інформації. Одним із ключових елементів безпеки є використання складних і надійних паролів для доступу до інформаційних систем. Однак створення таких паролів вручну часто є складним і незручним для користувачів, що призводить до використання слабких або повторюваних паролів, що значно підвищує ризик компрометації облікових записів. Тому розробка інструментів для автоматичної генерації надійних паролів є надзвичайно актуальною і важливою задачею.

Об'єктом дослідження є процеси автоматичної генерації та управління паролями у сучасних інформаційних системах. Предметом дослідження є методи та алгоритми генерації паролів, їх інтеграція у клієнт-серверну архітектуру, а також розробка REST API та веб-інтерфейсу для користувачів.

Метою даної курсової роботи є розробка програмного забезпечення, яке забезпечить автоматичну генерацію надійних паролів із різними параметрами безпеки та зручний інтерфейс для користувачів. Для досягнення цієї мети поставлено наступні завдання:

- Проаналізувати існуючі підходи до генерації паролів та вимоги до їх безпеки;
- Спроекувати та реалізувати модуль генерації паролів із підтримкою різних алгоритмів;
- Розробити веб-інтерфейс користувача для генерації, збереження та управління паролями;
- Створити REST API для інтеграції з іншими сервісами та клієнтськими застосунками;
- Інтегрувати Swagger UI для автоматичної документації та тестування API;
- Забезпечити безпеку доступу до функціоналу через механізми аутентифікації.

Таким чином, розроблена система забезпечить ефективне та безпечне управління паролями, що сприятиме підвищенню загального рівня безпеки.

1 АНАЛІЗ ПРОБЛЕМИ ТА ВИБІР ТЕХНОЛОГІЙ

1.1 Проблема генерації безпечних паролів

У сучасному цифровому світі паролі залишаються основним засобом захисту персональних даних та облікових записів. Вони використовуються для аутентифікації користувачів у різних системах, від соціальних мереж до банківських сервісів. Однак, незважаючи на важливість паролів, багато користувачів нехтують їхньою безпекою, що призводить до численних кіберзагроз.[1]

Згідно з дослідженнями, значна частина людей використовує слабкі або повторювані паролі, що робить їх вразливими до атак. Крім того, з розвитком технологій хакери отримують доступ до більш потужних методів злому, таких як атаки перебором, словникові атаки та соціальна інженерія. Це створює необхідність розробки ефективних алгоритмів генерації паролів, які забезпечують високий рівень безпеки.

Основні проблеми, пов'язані з генерацією безпечних паролів:

1. Використання слабких паролів. Багато користувачів обирають прості паролі, такі як “123456”, “password” або “qwerty”, які легко вгадати. Це робить їх вразливими до атак перебором (Brute Force), коли зловмисники автоматично підбирають комбінації символів.

2. Повторне використання одного пароля. Користувачі часто застосовують один і той самий пароль для кількох облікових записів. Якщо один із них буде зламаний, це може поставити під загрозу всі інші акаунти. Це особливо небезпечно у випадку витоку даних з великих платформ.

3. Складність запам'ятовування. Надійні паролі повинні містити різноманітні символи (цифри, літери різного регістру, спеціальні символи), що робить їх важкими для запам'ятовування. Через це користувачі часто записують паролі у незахищених місцях або використовують занадто прості комбінації.

4. Атаки перебором (Brute Force) та словникові атаки. Хакери використовують автоматизовані методи для злому паролів, що робить прості

комбінації вкрай ненадійними. Словникові атаки використовують бази даних поширених паролів, що значно скорочує час злому.

5. Фішингові атаки. Зловмисники створюють шахрайські вебсайти або надсилають електронні листи, що імітують офіційні повідомлення, змушуючи користувачів вводити свої паролі. Це одна з найпоширеніших загроз, яка дозволяє отримати доступ до облікових записів без необхідності злому.

6. Використання особистих даних у паролях. Багато людей включають у паролі свої імена, дати народження або номери телефонів, що робить їх легкими для вгадування. Соціальна інженерія дозволяє зловмисникам отримати ці дані та використовувати їх для підбору паролів.

7. Збереження паролів у незахищених місцях. Користувачі часто записують паролі у текстових файлах, нотатках на телефоні або навіть на папері. Це створює ризик їхнього викрадення сторонніми особами.

8. Відсутність двофакторної аутентифікації (2FA). Двофакторна аутентифікація додає додатковий рівень захисту, вимагаючи підтвердження через SMS, електронну пошту або спеціальні додатки.

9. Використання застарілих паролів. Паролі, які не змінюються протягом тривалого часу, можуть бути скомпрометовані через витоки даних або атаки. Регулярне оновлення паролів допомагає зменшити ризик їхнього використання зловмисниками.

10. Передача паролів стороннім особам

Деякі користувачі передають свої паролі друзям, колегам або навіть записують їх у загальнодоступних місцях. Це створює ризик несанкціонованого доступу.

Перспективи вирішення проблеми

Для підвищення рівня безпеки паролів необхідно впроваджувати сучасні методи їхньої генерації та зберігання. Серед ефективних рішень можна виділити:

- Використання менеджерів паролів – автоматичне створення та збереження складних паролів.
- Двофакторна аутентифікація (2FA) – додатковий рівень

захисту, що вимагає підтвердження через SMS або спеціальні додатки.[2]

- Біометрична аутентифікація – використання відбитків пальців або розпізнавання обличчя як альтернативи паролів.
- Криптографічні ключі та автентифікація без паролів (Passkeys) – новітні технології, що дозволяють входити в систему без необхідності запам'ятовувати складні комбінації.

Генерація безпечних паролів є критично важливим аспектом кібербезпеки. Використання складних комбінацій, унікальних паролів для кожного акаунта, двофакторної аутентифікації та сучасних методів аутентифікації допомагає значно знизити ризики. Крім того, підвищення обізнаності користувачів щодо кібербезпеки є ключовим фактором у боротьбі з загрозами.

1.2 Аналіз існуючих рішень

У сучасному цифровому світі захист паролів є критично важливим аспектом кібербезпеки.[2] Для збереження та управління паролями користувачі все частіше звертаються до спеціалізованих менеджерів паролів. Серед найпопулярніших рішень можна виділити KeePass, LastPass, Bitwarden, 1Password та Dashlane. Кожен із цих менеджерів має свої особливості, переваги та недоліки. Розглянемо кожен з них:

KeePass — це локальний менеджер паролів з відкритим кодом, який дозволяє зберігати паролі у зашифрованій базі даних. Його основні характеристики:

- Безпека: Використовує 256-бітне AES-шифрування, що забезпечує високий рівень захисту.
- Гнучкість: Підтримує плагіни, які розширюють функціональність (наприклад, автозаповнення паролів у браузері).
- Локальне зберігання: Паролі зберігаються офлайн, що виключає ризик витоку даних через хмарні сервіси.
- Недоліки: Відсутність автоматичної синхронізації між пристроями та складний інтерфейс для новачків.

LastPass — комерційний хмарний менеджер паролів, який пропонує зручний доступ до паролів з будь-якого пристрою. Основні особливості:

- Зручність: Автоматичне заповнення паролів у браузерях та додатках.
- Синхронізація: Паролі зберігаються у хмарі, що дозволяє отримати доступ до них з будь-якого пристрою.
- Двофакторна аутентифікація (2FA): Додатковий рівень захисту для входу в акаунт.
- Недоліки: Уразливість до витоків даних (у минулому компанія стикалася з проблемами безпеки) та висока вартість преміум-підписки.

Bitwarden — мультиплатформний менеджер паролів з відкритим кодом, який поєднує безпеку та зручність. Його ключові характеристики:

- Відкритий код: Дозволяє незалежним експертам перевіряти безпеку програми.
- Синхронізація: Підтримує хмарне зберігання паролів із можливістю локального використання.
- Автозаповнення: Інтеграція з браузерами для швидкого введення паролів.
- Недоліки: Вимагає інтернет-з'єднання для синхронізації та має обмежені можливості кастомізації у порівнянні з KeePass.

1Password — популярний комерційний менеджер паролів, який пропонує розширені функції безпеки:

- Шифрування: Використовує AES-256 для захисту даних.
- Watchtower: Функція моніторингу витоків паролів.
- Підтримка біометрії: Вхід за допомогою відбитка пальця або Face ID.
- Недоліки: Висока вартість підписки та відсутність безкоштовної версії.

Dashlane — ще один комерційний менеджер паролів, який пропонує:

- Автоматичне заповнення паролів у браузерах.

- VPN для захисту під час використання незахищених мереж.
- Моніторинг темного вебу для перевірки витоків паролів.
- Недоліки: Висока ціна та обмежена безкоштовна версія.

Вибір менеджера паролів залежить від потреб користувача. KeePass підходить для тих, хто хоче повний контроль над паролями без використання хмарних сервісів. Bitwarden є золотою серединою між безпекою та зручністю. LastPass, 1Password та Dashlane пропонують розширені функції, але вимагають підписки.

1.3 Обґрунтування вибору технологій

Для розробки програмного засобу було обрано набір перевірених та сучасних технологій, які забезпечують ефективну реалізацію функціоналу, зручність тестування, супроводу, а також розширення у майбутньому.

Java 17 — як основна мова програмування. Вона є надійною, об'єктно-орієнтованою та широко підтримуваною, з великою кількістю бібліотек і фреймворків. Java добре підходить для створення веб-застосунків серверної частини.

Spring Boot — фреймворк, що значно спрощує створення веб-додатків, автоматизуючи конфігурацію компонентів. Він дозволяє швидко запускати проєкт, має вбудований вебсервер (Tomcat), інтеграцію з базами даних, безпекою, REST API тощо.

HTML5, CSS (Bootstrap) — для створення адаптивного, сучасного та візуально привабливого інтерфейсу. Bootstrap дозволяє швидко стилізувати елементи, дотримуючись принципів адаптивного дизайну.

JavaScript (Fetch API) — використовується для реалізації асинхронних запитів до серверу, зокрема для динамічного видалення паролів без перезавантаження сторінки. Це покращує взаємодію з користувачем.

Spring Data JPA — модуль Spring для роботи з базами даних, що забезпечує просту реалізацію CRUD-операцій без написання SQL-запитів. Це

підвищує продуктивність розробки та покращує читабельність коду.

PostgreSQL — потужна об'єктно-реляційна база даних з відкритим кодом, яка широко використовується для розробки веб-додатків, корпоративних систем та аналітичних рішень. Вона підтримує складні запити, транзакції та реплікацію, що робить її чудовим вибором для великих та середніх проєктів. PostgreSQL пропонує розширені можливості роботи з типами даних, включаючи JSON та XML, а також має вбудовані засоби для оптимізації продуктивності. Її встановлення потребує більше налаштувань у порівнянні з H2, але вона забезпечує значно вищу гнучкість та надійність. PostgreSQL має активну спільноту, що постійно вдосконалює функціонал та виправляє помилки. Вона також пропонує потужний механізм розширень, що дозволяє додавати нові можливості без необхідності змінювати основний код. Завдяки своїй стабільності та масштабованості, PostgreSQL часто використовується у великих комерційних продуктах та складних обчислювальних системах.

Thymeleaf — серверний шаблонізатор HTML, що забезпечує динамічне відображення даних на сторінках. Завдяки глибокій інтеграції з Spring MVC він дозволяє зручно формувати інтерфейс користувача без потреби в сторонніх JavaScript-фреймворках.

Swagger UI (Springdoc OpenAPI) — інтегрований для автоматичної генерації документації REST API. Swagger надає зручний веб-інтерфейс для перевірки запитів, перегляду структури об'єктів, що дозволяє спростити тестування та взаємодію з API.

JUnit 5 — використовується для модульного тестування логіки генерації паролів та взаємодії з базою даних. JUnit дозволяє переконатися у правильності роботи окремих компонентів і підвищує надійність системи.

Таким чином, обраний набір технологій є оптимальним для реалізації функціонального, надійного та зручного веб-застосунку. Усі компоненти добре інтегруються між собою, мають активну спільноту, документацію та дозволяють легко масштабувати додаток у майбутньому.

2 ПРОЕКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

2.1 Функціональні вимоги

Розроблене програмне забезпечення є веб-додатком для генерації паролів, яке дозволяє користувачам отримувати надійні паролі за різними алгоритмами генерації, а також зберігати їх у базі даних після авторизації. Система підтримує як роботу для гостей (неавторизованих користувачів), так і для зареєстрованих користувачів.

Основні функції:

1. Генерація пароля для гостя

- Користувач, який не проходив авторизацію, може згенерувати пароль.
- Можна вибрати довжину пароля.
- Можна обрати тип алгоритму генерації (ентропійний, passphrase, рандомний).
- Можна вказати, чи включати символи та цифри.
- Згенерований пароль відображається на екрані.
- Пароль не зберігається в базі даних.

2. Авторизація на сайті

- Користувач може авторизуватися за допомогою облікового запису Google.
- Після успішної авторизації зберігається інформація про користувача (ім'я, прізвище, email).
- Користувач отримує доступ до персонального профілю.

3. Генерація пароля для авторизованого користувача

- Користувач може генерувати паролі з тими ж параметрами, що і гість.
- Згенерований пароль автоматично зберігається в базі даних, прив'язаний до профілю користувача.
- Зберігаються параметри пароля: довжина, тип генерації, дата створення.

4. Перегляд історії паролів

- Авторизований користувач може переглядати таблицю зі всіма збереженими паролями.

- У таблиці відображаються параметри пароля: текст, тип, довжина, дата створення і останнього оновлення.

5. Редагування паролів

- Користувач може оновити текст пароля в збережених записах.
- При оновленні зберігається дата останньої модифікації.
- Довжина пароля автоматично оновлюється відповідно до нового тексту.

6. Видалення паролів

- Користувач може видаляти будь-який зі своїх збережених паролів.
- Після видалення пароль зникає з бази даних і більше не відображається в історії.

7. REST API для роботи з паролями

- Система надає REST API для отримання списку паролів, створення, оновлення та видалення паролів.
- API підтримує стандартні HTTP методи: GET, POST, PUT, DELETE.
- Доступ до API контролюється авторизацією.

Додаткові вимоги:

- Веб-інтерфейс повинен бути інтуїтивним і зручним для користувача.
- Паролі генеруються за допомогою криптографічно стійких алгоритмів.
- Система має підтримувати кілька типів генерації паролів для підвищення безпеки.
- Дані користувача і паролі зберігаються у захищеній базі даних.
- Всі операції зі збереження, оновлення і видалення паролів мають логуватися для відстеження.

На діаграмі Use Case (Рисунок 2.1) представлені два основні актори: **Гість** та **Авторизований користувач**. Вони мають різні рівні доступу до функціональності системи, що відображає особливості їх взаємодії з додатком.

Гість — це користувач, який ще не пройшов авторизацію. Йому доступні базові дії, зокрема можливість авторизуватися в системі. Крім того, гість може згенерувати пароль, обираючи при цьому параметри, такі як довжина пароля, використання літер і символів, а також метод генерації пароля. Ця функція

дозволяє отримати пароль без збереження його у базі даних.

Авторизований користувач має розширений набір можливостей. Він може виконувати всі дії, доступні гостю, а також додатково переглядати вже створені паролі. Окрім цього, авторизований користувач може управляти паролями — зберігати нові паролі в базі даних, оновлювати існуючі та видаляти непотрібні. Також у його функціонал входить можливість вийти з облікового запису.

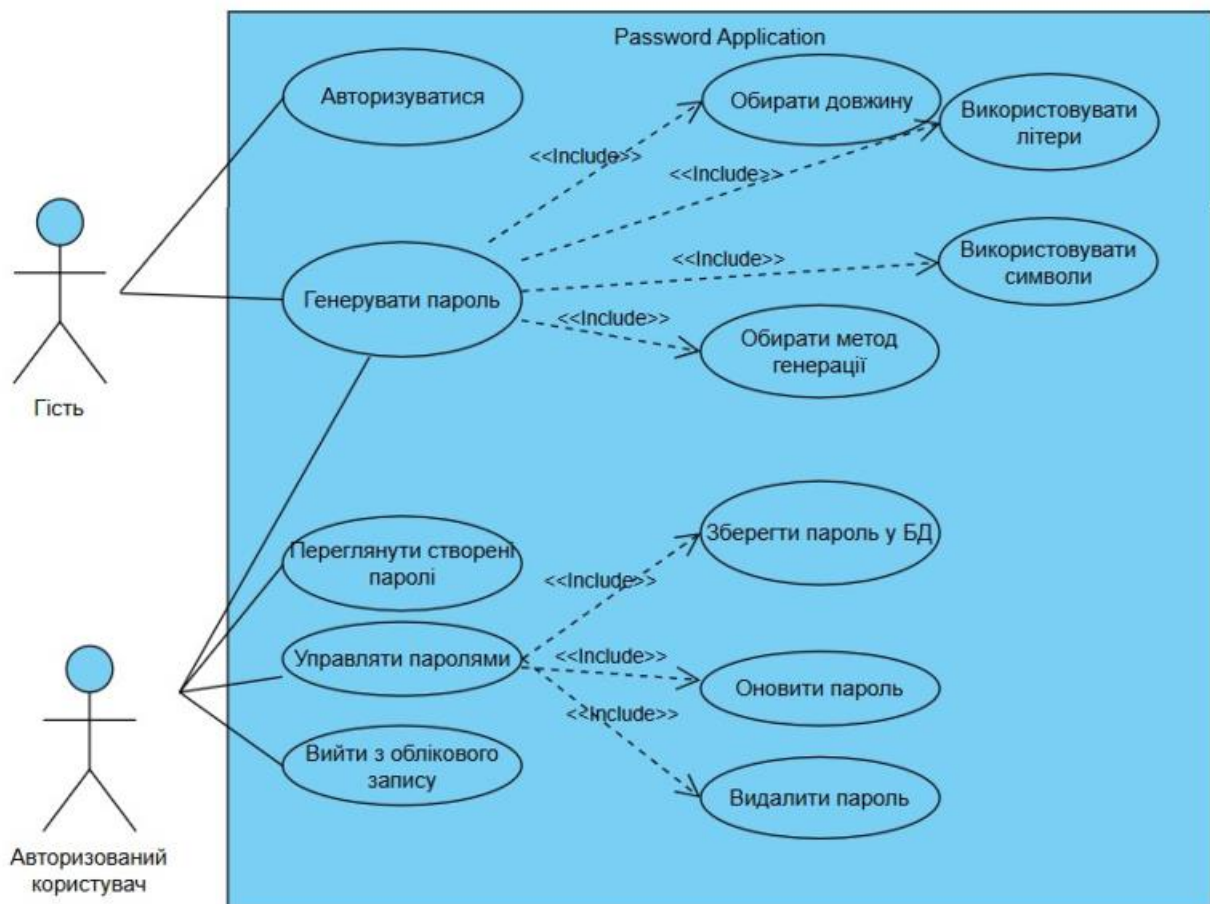


Рисунок 2.1 — UseCase - діаграма

У діаграмі використовується зв'язок «include», який показує, що вибір параметрів генерації пароля є частиною процесу його створення. Аналогічно, операції зі збереження, оновлення та видалення паролів входять до складу управління паролями. Такий підхід дозволяє структуровано відобразити взаємозв'язки між функціями системи.

Загалом, дана Use Case діаграма наочно демонструє ключові функції

системи для різних типів користувачів. Вона слугує основою для формування вимог до розробки та розуміння того, як користувачі взаємодіють із додатком залежно від їхньої ролі.

2.2 Нефункціональні вимоги

Нефункціональні вимоги визначають якісні характеристики системи, які забезпечують її ефективність, надійність, безпеку, зручність використання та підтримуваність. Вони доповнюють функціональні вимоги та безпосередньо впливають на архітектуру, реалізацію і подальшу експлуатацію системи.

Технологічні вимоги. Інтерфейс користувача повинен бути адаптивним і інтуїтивно зрозумілим. Для цього використовується сучасний стек технологій: HTML5, CSS із фреймворком Bootstrap для швидкого створення адаптивного дизайну, а також Thymeleaf як серверний шаблонізатор для динамічного формування веб-сторінок.

Серверна логіка реалізована на основі Java 17 з використанням Spring Boot, що дозволяє побудувати масштабований та підтримуваний бекенд. Збереження даних здійснюється у реляційній базі даних, що забезпечує структуроване зберігання інформації про користувачів та їхні паролі з можливістю виконання складних запитів.

Для забезпечення чіткого розділення бізнес-логіки та інтерфейсу користувача реалізовано REST API. Це дозволяє досягти масштабованості системи та відкриває можливості для подальшої інтеграції з іншими сервісами або клієнтськими додатками.

Документація API генерується автоматично за допомогою Swagger UI, що значно полегшує розуміння, тестування та підтримку API.

Продуктивність та динамічність. Веб-додаток повинен відгукуватися на дії користувача протягом 1-2 секунд при типовому навантаженні. Оптимізація запитів до бази даних та застосування кешування мають гарантувати швидке виконання операцій читання і запису.

Безпека. Система реалізує безпечну аутентифікацію користувачів через

OAuth2 з провайдером Google. Паролі користувачів зберігаються у базі у захищеному вигляді, використовуючи сучасні методи шифрування або хешування.

Для захисту від типових веб-атак, таких як XSS, CSRF та SQL-ін'єкції, впроваджені відповідні заходи безпеки. Всі дані передаються по захищеному протоколу HTTPS, що забезпечує конфіденційність інформації.

Система дотримується нормативних вимог щодо захисту персональних даних користувачів.

Зручність використання (Usability). Інтерфейс додатку повинен бути простим у навігації та зрозумілим як для гостей, так і для авторизованих користувачів. Повідомлення про успіх або помилки операцій відображаються у вигляді сповіщень, що підвищує зручність взаємодії.

Веб-додаток підтримує коректну роботу на різних пристроях і розмірах екранів завдяки адаптивному дизайну. Окрім того, передбачена підтримка української мови інтерфейсу з можливістю розширення локалізації.

Надійність та підтримуваність. Важливі дані регулярно резервуються для запобігання їх втраті. Система стійка до помилок і збоїв, а у випадку аварій має швидко відновлювати роботу без втрати інформації.

Для подальшого аналізу ведеться журналювання помилок і подій. Архітектура коду модульна, що сприяє легкій підтримці і розширенню функціональності.

Покриття критичних частин системи модульними та інтеграційними тестами на базі JUnit 5 гарантує високу якість і стабільність роботи програмного забезпечення.

Сумісність та портативність. Додаток підтримує роботу у популярних браузерах, таких як Chrome, Firefox, Edge та Safari. Платформонезалежність забезпечує доступність системи для користувачів на різних операційних системах, включно з Windows, macOS, Linux, Android та iOS.

2.3 Проектування БД

Для реалізації системи генерації і збереження паролів було спроектовано три основні таблиці в базі даних, що відображають ключові сутності додатку: користувачі, типи паролів та самі паролі (Рисунок 2.2).

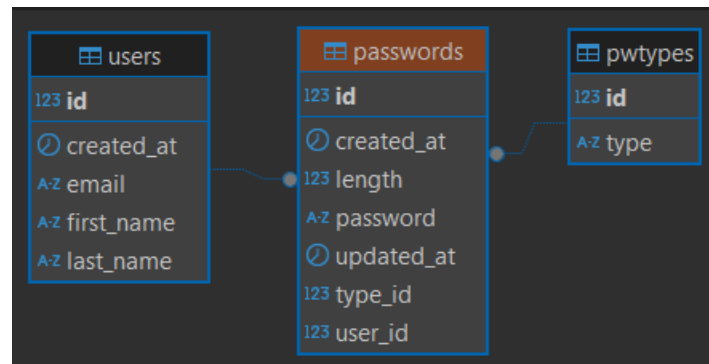


Рисунок 2.2 — Структура БД

Таблиця Users. Ця таблиця містить інформацію про користувачів системи. Кожен користувач має унікальний ідентифікатор `id`, а також особисті дані: ім'я (`firstName`), прізвище (`lastName`) та електронну пошту (`email`). Крім того, зберігається дата створення запису `createdAt`. Таблиця є основою для ідентифікації користувачів, які можуть створювати та управляти паролями.

Таблиця PWTypes. Дана таблиця відповідає за зберігання типів паролів, які можуть бути згенеровані системою. Кожен тип має унікальний ідентифікатор `id` та назву типу `type`. Ця таблиця допомагає класифікувати паролі відповідно до алгоритму створення і використовується для зв'язку з таблицею паролів.

Таблиця Passwords. Таблиця зберігає безпосередньо інформацію про згенеровані паролі. Кожен запис має унікальний ідентифікатор `id`, текст пароля `password`, а також довжину пароля `length`. Для кожного пароля вказується тип через зовнішній ключ `type_id`, який посилається на таблицю `PWTypes`, і користувач через зовнішній ключ `user_id`, який посилається на таблицю `Users`. Також у таблиці фіксуються дати створення (`createdAt`) та останнього оновлення (`updatedAt`) запису. Завдяки цьому забезпечується історія та контроль за паролями, створеними користувачами.

Ця структура бази даних забезпечує ефективне зберігання та управління інформацією про користувачів, типи паролів і самі паролі, підтримуючи

цілісність даних і дозволяючи реалізувати функціонал додатку згідно з вимогами.

2.4 User Stories та діаграма послідовності

Для реалізації функціоналу генерації та збереження пароля були визначені наступні користувацькі історії (User Stories):

User story 1: Як зареєстрований користувач, я хочу надіслати запит на генерацію пароля з параметрами (довжина, тип, символи, цифри), щоб отримати надійний пароль для захисту своїх акаунтів.

Критерії прийняття:

- Користувач може вказати параметри генерації пароля.
- Система приймає запит і генерує пароль відповідно до заданих параметрів.

User story 2: Як система, я хочу отримати дані користувача (email, ім'я, прізвище) через OAuth2, щоб зв'язати згенерований пароль із конкретним користувачем.

Критерії прийняття:

- Система успішно отримує та підтверджує дані користувача з OAuth2.
- Дані використовуються для подальшої роботи з паролем.

User story 3: Як сервіс генерації паролів, я хочу створювати пароль з урахуванням параметрів користувача, щоб пароль відповідав вимогам безпеки і був зручним у використанні.

Критерії прийняття:

- Пароль генерується динамічно з урахуванням усіх параметрів.
- Пароль є унікальним та безпечним.

User story 4: Як сервіс збереження паролів, я хочу зберігати згенерований пароль у базі даних, щоб користувач міг потім його переглянути або використати.

Критерії прийняття:

- Пароль зберігається у базі даних.
- Система підтверджує успішне збереження.

User story 5: Як користувач, я хочу бачити згенерований пароль разом із моїми профільними даними після генерації, щоб швидко скопіювати та використовувати його.

Критерії прийняття:

- Після генерації та збереження пароля користувачу відображаються пароль та профільні дані.
- Інтерфейс працює швидко і без помилок.

User story 6: Як зареєстрований користувач, я хочу надіслати запит на генерацію пароля з параметрами (довжина, тип, символи, цифри), щоб отримати надійний пароль, який система згенерує, збереже та відобразить разом із моїми профільними даними.

Критерії прийняття:

- Відправку запиту з параметрами
- Отримання профільних даних через OAuth2
- Виклик сервісу генерації пароля
- Повернення згенерованого пароля назад до контролера
- Збереження пароля у базі
- Підтвердження збереження
- Відображення пароля та профільних даних користувачу

Діаграма послідовності(Рисунок 2.3) ілюструє **User story 6**, процес генерації пароля користувачем, який пройшов авторизацію, із подальшим збереженням згенерованого пароля у базі даних. Дана діаграма складається з:

1. Ініціація дії користувачем: Користувач відправляє HTTP POST-запит на контролер MainController за маршрутом /generatebyuser, передаючи параметри: довжину пароля, тип генерації, а також вказівки, чи включати символи і цифри.

2. Отримання профільних даних користувача: Контролер отримує з об'єкта OAuth2User інформацію про користувача — email, ім'я та прізвище, які потрібні для зв'язування пароля з користувачем у системі.

3. Генерація пароля: Контролер викликає сервіс PasswordService, який у свою чергу делегує завдання генерації відповідному класу генератора паролів (PasswordGenerator) згідно вибраного типу. Генератор повертає згенерований пароль.

4. Збереження пароля: Згенерований пароль разом із додатковими метаданими (користувач, тип, довжина, час створення) передається назад у PasswordService для збереження. Сервіс зберігає об'єкт у базі даних через репозиторій PasswordRepository.

5. Повернення результату користувачу: Після успішного збереження пароля контролер повертає користувачу відповідь із новим паролем та його профільною інформацією, яку можна відобразити на інтерфейсі.

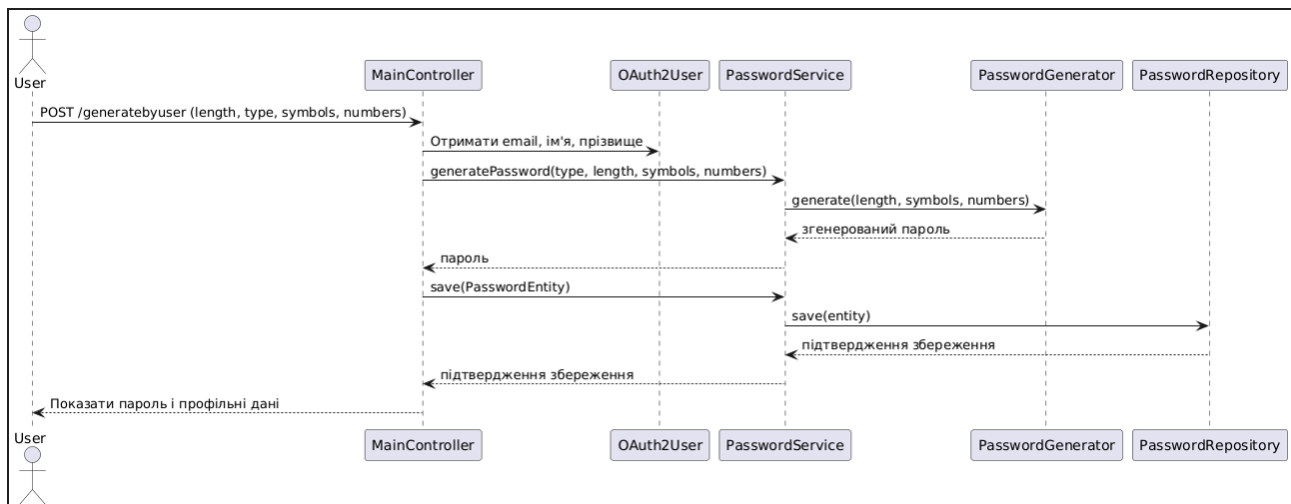


Рисунок 2.3 — Діаграма послідовності

Ця діаграма демонструє ключові взаємодії між користувачем, веб-шаром (контролером), бізнес-логікою (сервісом), механізмом генерації паролів і шаром доступу до даних (репозиторієм). Вона ілюструє послідовність кроків та обмін повідомленнями, необхідними для успішного виконання функції генерації та збереження пароля в системі.

2.5 Алгоритми генерації паролів

У рамках даного програмного забезпечення необхідно реалізувати

підтримку трьох основних алгоритмів генерації паролів. Кожен з них має свої переваги та підходить для різних сценаріїв використання, враховуючи вимоги до безпеки, довжини та зручності.

2.5.1 Random

Даний алгоритм базується на використанні криптографічно безпечного генератора випадкових чисел (наприклад, SecureRandom у Java). Символи для пароля обираються випадковим чином із заданого набору: великі та малі літери, цифри, а також спеціальні символи. Такий метод забезпечує непередбачуваність і складність отриманого пароля. Паролі, згенеровані цим способом, не мають очевидної структури, що ускладнює їхнє вгадування або перебір. Цей тип генерації рекомендований для стандартного використання в більшості систем.

2.5.2 Entropy-based

Алгоритм генерації з високою ентропією фокусується на максимально можливій випадковості, з урахуванням розподілу символів і уникнення шаблонних комбінацій. Він також використовує SecureRandom, але обчислення ґрунтується на принципах інформаційної ентропії: кожен символ обирається так, щоб максимально збільшити непередбачуваність всієї послідовності. Цей підхід особливо корисний у середовищах з підвищеними вимогами до безпеки (наприклад, корпоративних або фінансових системах). Результат — складний, нерегулярний пароль, стійкий до атак типу brute-force.

2.5.3 Passphrase

Алгоритм "Passphrase" (фразовий пароль) будує пароль не з окремих символів, а з кількох слів або частин слів, які генеруються за допомогою словника. Такий підхід дозволяє створювати довші, але більш запам'ятовувані паролі. Часто використовуються техніки випадкового поєднання іменників, дієслів та прикметників для досягнення як складності, так і зручності. Наприклад, пароль типу happy-Horse!River9 одночасно складний для

автоматичного злому, але досить зручний для запам'ятовування користувачем. Цей метод поєднує безпеку та людську зручність.

3 РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Структура проекту та опис модулів системи

Система реалізована за принципом клієнт-серверної архітектури. В якості серверної частини використовується фреймворк Spring Boot, який відповідає за обробку HTTP-запитів, бізнес-логіку та взаємодію з базою даних. Для рендерингу HTML-інтерфейсу застосовується шаблонізатор Thymeleaf, що інтегрується з Spring MVC. Обмін даними між клієнтом і сервером відбувається за допомогою REST API, що забезпечує гнучкість і масштабованість системи.

1. Основні модулі системи:

DemoApplication.java – головний клас, що запускає Spring Boot-застосунок. Містить метод `main`, з якого стартує програма.

OpenApiConfig.java – конфігураційний клас для налаштування OpenAPI/Swagger-документації REST API, що використовується для тестування та документування інтерфейсів.

SecurityConfig.java – клас налаштування безпеки, відповідальний за обробку аутентифікації користувачів через OAuth2 та захист доступу до певних маршрутів.

2. Контролери:

MainController.java – контролер, що реалізує логіку веб-інтерфейсу для користувачів. Він обробляє запити до головної сторінки, генерує паролі на основі введених параметрів (довжина, тип, символи, цифри), дозволяє зберігати паролі для авторизованих користувачів через OAuth2. Окрім генерації, `MainController` також надає функціонал для перегляду історії створених паролів, редагування, видалення записів та відображення профілю користувача разом з даними Google-акаунту (ім'я, прізвище, email, аватар). Дані зберігаються у базу даних за допомогою `PasswordService`.

PasswordController.java – REST-контролер, який надає API для взаємодії з об'єктами паролів. Реалізує повний CRUD-функціонал:

- Отримання списку всіх паролів.
- Отримання конкретного пароля за ID.
- Створення нового пароля.
- Оновлення пароля.
- Видалення пароля.

Контролер анотований за допомогою OpenAPI/Swagger, що дозволяє автоматично генерувати документацію для API. Це забезпечує зручність при тестуванні, розробці та інтеграції з іншими системами. REST-контролер розрахований на використання зовнішніми клієнтами або фронтенд-застосунками, які потребують взаємодії через JSON.

3. Сутності (Entity-класи):

PasswordEntity.java – описує модель пароля, включаючи його значення, довжину, тип, дату створення та останнього оновлення, а також зв'язки з користувачем і типом пароля.

PWTypeEntity.java – сутність, що представляє тип пароля (наприклад, випадковий, фразовий, ентропійний).

UserEntity.java – зберігає інформацію про користувачів: ім'я, прізвище, email, дату реєстрації. Пов'язується з паролями, створеними цим користувачем.

4. Генератори паролів:

EntropyBasedPasswordGenerator.java – реалізує алгоритм генерації паролів з урахуванням ентропії, що забезпечує високу стійкість до атак.

PassphrasePasswordGenerator.java – створює пароль на основі фраз, зручних для запам'ятовування.

RandomPasswordGenerator.java – генерує випадковий набір символів згідно з обраними параметрами.

PasswordGenerator.java – загальний інтерфейс або абстракція для різних реалізацій генераторів.

5. Репозиторії (інтерфейси доступу до БД):

PasswordRepository.java – інтерфейс доступу до таблиці паролів. Відповідає за пошук, збереження, оновлення і видалення паролів через Spring

Data JPA.

PWTypeRepository.java – репозиторій для роботи з типами паролів.

UserRepository.java – забезпечує доступ до даних користувачів.

6. Сервіси (бізнес-логіка):

PasswordService.java – головний сервіс генерації та збереження паролів.

Об'єднує логіку вибору типу генерації, взаємодіє з генераторами та репозиторієм.

PWTypeService.java – сервіс для роботи з типами паролів, включаючи ініціалізацію та доступ до них.

UserService.java – обробляє бізнес-логіку, пов'язану з користувачами, зокрема створення та пошук за email.

Завдяки чіткому поділу на контролери, сервіси, генератори, сутності та репозиторії система є модульною, зручною для підтримки, тестування і розширення.

3.2 Інтерфейс користувача

Інтерфейс користувача реалізовано за допомогою шаблонізатора Thymeleaf у рамках архітектури Spring MVC. Це забезпечує динамічне формування HTML-сторінок на стороні сервера з використанням змінних та моделей, переданих з контролерів. Користувацький інтерфейс є простим, інтуїтивно зрозумілим і дозволяє зручно взаємодіяти з основними функціями системи.

Основні екрани:

Головна сторінка (home.html) – відображає форму генерації пароля. Користувач (в тому числі неавторизований) може ввести параметри (довжину, тип генерації, включення символів і цифр) та згенерувати пароль. Результат виводиться на тій же сторінці (Рисунок 3.1).

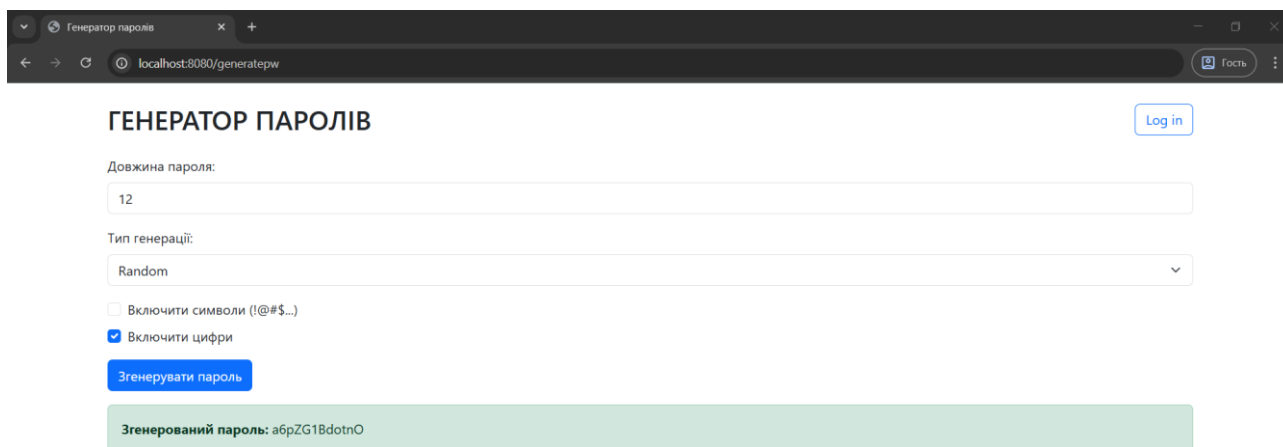


Рисунок 3.1 — Головна сторінка

Сторінка користувача (profile.html) – доступна лише авторизованим користувачам (через Google OAuth2). Виводить особисту інформацію користувача, таку як ім'я, прізвище, електронну пошту, а також аватар, отриманий з Google (Рисунок 3.2).

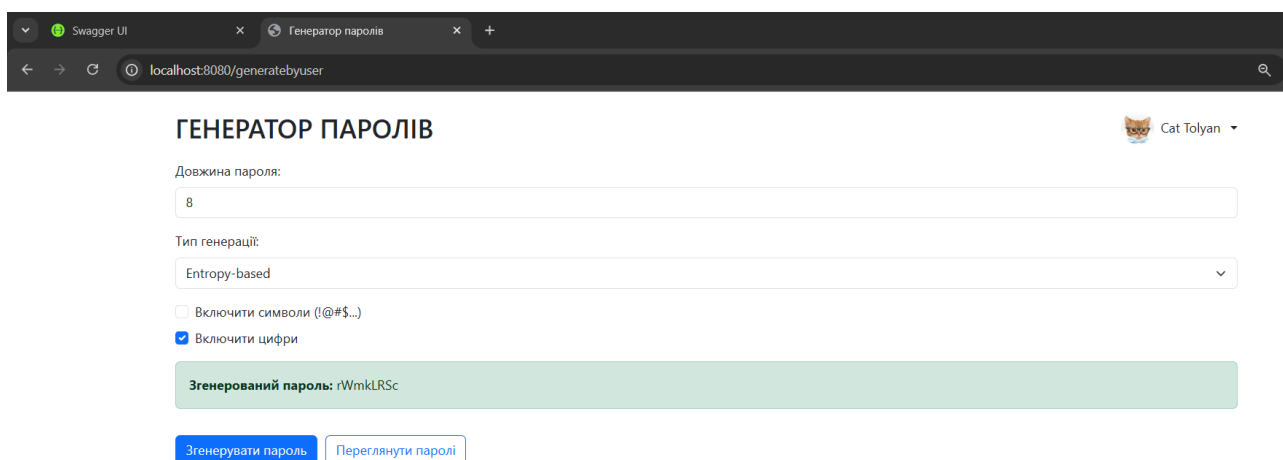
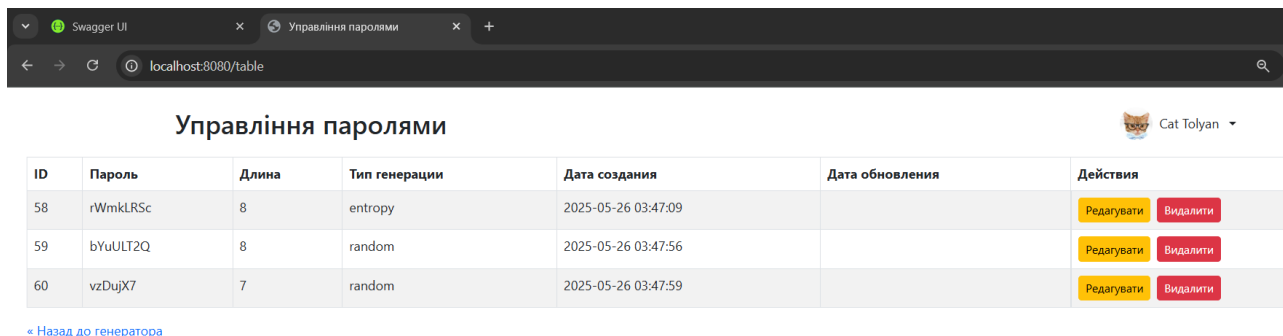


Рисунок 3.2 — Сторінка авторизованого користувача

Історія паролів (table.html) – відображає таблицю з усіма паролями, які були створені та збережені цим користувачем. У таблиці є кнопки для редагування та видалення паролів (Рисунок 3.3).

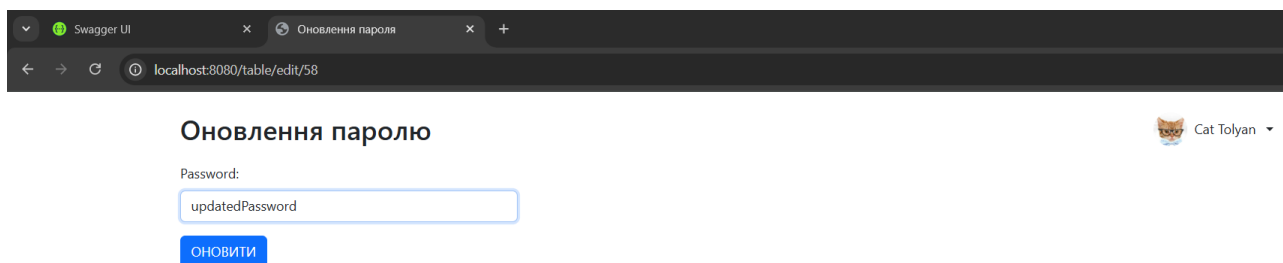


ID	Пароль	Длина	Тип генерации	Дата создания	Дата обновления	Действия
58	rWmkLRSc	8	entropy	2025-05-26 03:47:09		Редагувати Видалити
59	bYuULT2Q	8	random	2025-05-26 03:47:56		Редагувати Видалити
60	vzDujX7	7	random	2025-05-26 03:47:59		Редагувати Видалити

[Назад до генератора](#)

Рисунок 3.3 — Управління пароллями

Форма редагування пароля (edit-password-form.html) – дозволяє змінити вміст обраного пароля(Рисунок 3.4).



Оновлення паролю

Password:

[ОНОВИТИ](#)

Рисунок 3.4 — Редагування пароля

Дана форма передає id пароля разом зі зміненим значенням форми в контролер, тому після внесення змін пароль оновлюється в базі даних.

3.3 REST API та підключення Swagger UI

У розробленій системі реалізовано REST API, який забезпечує доступ до функціоналу збереження, оновлення, перегляду та видалення згенерованих паролів. REST-архітектура дозволяє виконувати всі операції над даними за допомогою стандартних HTTP-запитів, що є зручним для розробників і дає змогу легко масштабувати систему або інтегрувати її з іншими сервісами.

Інтерфейс REST API реалізовано в окремому контролері PasswordController, який відповідає за обробку запитів, пов'язаних із роботою паролів. Основні ендпоїнти API мають наступне призначення:

- GET /api/passwords — повертає список усіх збережених паролів;
- GET /api/passwords/{id} — повертає конкретний пароль за його ідентифікатором;
- POST /api/passwords — створює новий запис пароля;
- PUT /api/passwords/{id} — оновлює існуючий пароль;
- DELETE /api/passwords/{id} — видаляє пароль із системи.

Кожен метод супроводжується відповідними анотаціями OpenAPI, які дозволяють автоматично формувати документацію та описувати поведінку ендпоінтів (наприклад, очікувані параметри, коди відповіді, типи даних тощо).

Для візуалізації та тестування REST API у веб-інтерфейсі інтегровано Swagger UI (Рисунок 3.5). Цей інструмент надає можливість взаємодіяти з API безпосередньо з браузера — надсилати запити, переглядати відповіді, перевіряти формат введення тощо. Swagger UI автоматично генерує документацію на основі OpenAPI-нотацій у коді, що значно полегшує тестування й інтеграцію.

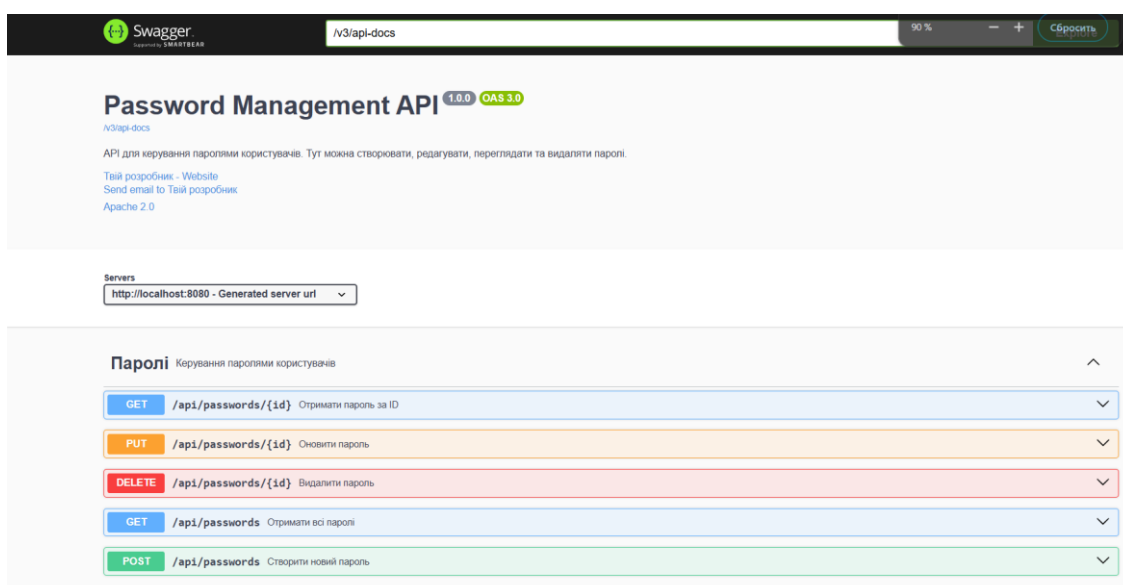


Рисунок 3.5 - Swagger UI

Використання Swagger UI забезпечує наступні переваги:

- зручне тестування API без необхідності застосування сторонніх інструментів;
- наочне представлення структури API;
- підвищення прозорості та доступності серверної логіки для

розробників;

- можливість швидко виявляти та усувати помилки в роботі API.

Таким чином, REST API разом зі Swagger UI забезпечує гнучкий, масштабований і добре документований механізм взаємодії з системою, що відповідає сучасним стандартам розробки веб-застосунків.

3.4 Тестування системи

Для забезпечення якості та коректної роботи модуля генерації паролів у системі було проведено комплексне тестування сервісу PasswordService. Тестування було реалізовано з використанням бібліотеки JUnit 5 та фреймворку Mockito для імітації залежностей.

Основною метою тестів було перевірити, що сервіс правильно генерує паролі з урахуванням заданих параметрів, таких як тип генерації, довжина паролю, а також наявність цифр та спеціальних символів.

Опис тестів:

Ініціалізація тестового середовища. Перед кожним тестом виконується налаштування об'єктів, де за допомогою Mockito створюється мок-репозиторій PasswordRepository, а сервіс PasswordService ініціалізується з трьома типами генераторів паролів: випадковий (RandomPasswordGenerator), фразовий (PassphrasePasswordGenerator) та генератор на основі ентропії (EntropyBasedPasswordGenerator).

Тестування генерації випадкового пароля (random). Перевіряється, що згенерований пароль (Рисунок 3.6):

- не є null, тобто генерація пароля відбувається успішно;
- має коректну довжину, що відповідає параметру запиту;
- містить принаймні одну цифру, якщо параметр includeDigits увімкнено;
- містить принаймні один спеціальний символ, якщо параметр includeSymbols увімкнено;
- не містить спеціальних символів, якщо параметр includeSymbols вимкнено;

- не містить цифр, якщо параметр `includeDigits` вимкнено.

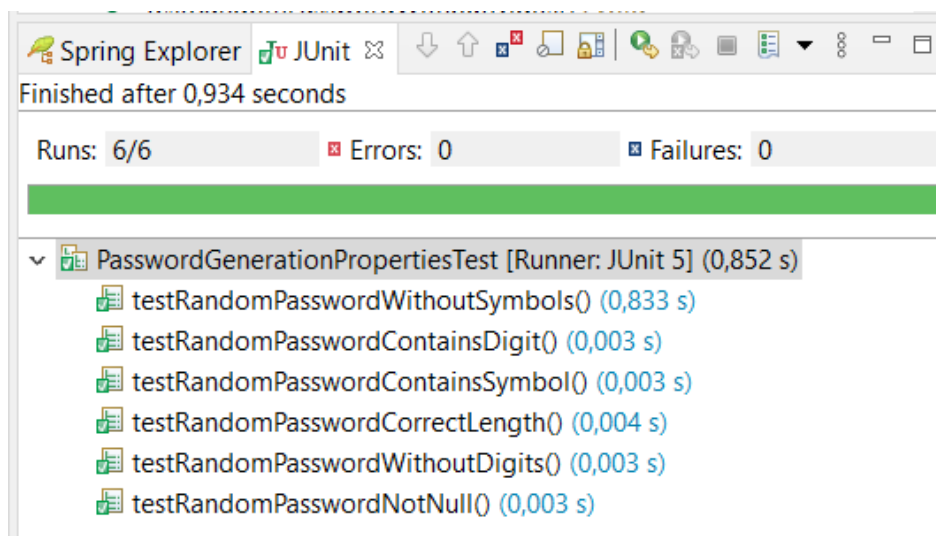


Рисунок 3.6 — Результати тестування

Ці тести гарантують, що сервіс генерації паролів коректно враховує налаштування безпеки та відповідає вимогам до паролів, які можуть бути необхідними для різних сценаріїв використання.

У даному проєкті реалізовано інтеграційні тести для контролера, який відповідає за роботу з паролями — `PasswordController`. Ці тести виконуються з метою перевірки правильності обробки HTTP-запитів і коректності взаємодії між веб-шаром додатку і базою даних (Рисунок 3.7).

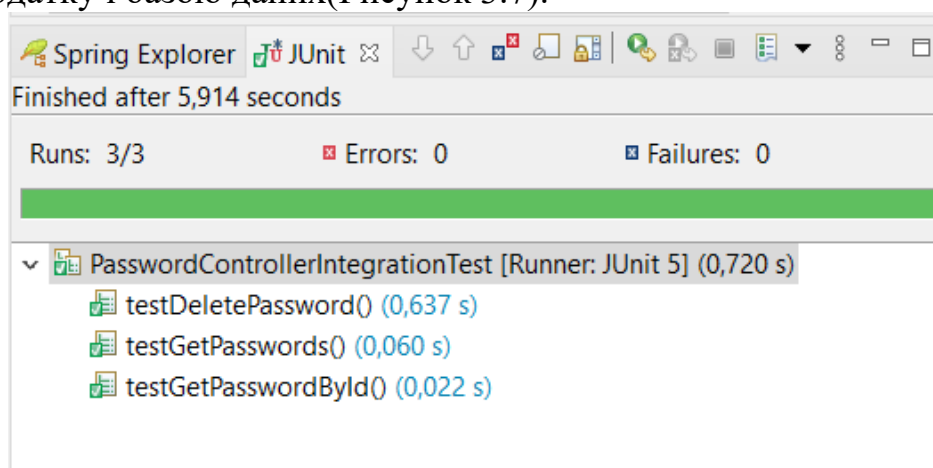


Рисунок 3.7 — Інтеграційне тестування

Перший тест перевіряє функціонал отримання всіх паролів. Для цього у

тестовій базі даних створюється один об'єкт пароля з конкретним значенням. Після цього відправляється GET-запит на адресу `/api/passwords`. Очікується, що сервер поверне успішний HTTP статус 200, а у тілі відповіді буде масив, який містить один запис з раніше створеним паролем. Цей тест підтверджує, що метод контролера для отримання списку паролів працює коректно і дані успішно повертаються клієнту.

Другий тест спрямований на перевірку можливості отримати конкретний пароль за його унікальним ідентифікатором. У базу даних додається новий пароль, після чого відправляється GET-запит за адресою `/api/passwords/{id}`, де `{id}` — це ідентифікатор створеного пароля. Тест очікує, що сервер поверне статус 200, а у відповіді міститиметься JSON-об'єкт з полем `password`, значення якого співпадає зі збереженим паролем. Цей тест гарантує, що контролер правильно обробляє запити на отримання окремого ресурсу за його ID.

Третій тест перевіряє видалення пароля. Для цього спочатку створюється пароль у базі даних, а потім виконується DELETE-запит за його ідентифікатором. Очікується, що сервер відповість статусом 204, що означає успішне видалення ресурсу без повернення контенту. Після цього виконується GET-запит за тим самим ID, щоб переконатися, що пароль дійсно видалено — у відповідь має повернутися статус 404 (ресурс не знайдено). Цей тест підтверджує, що операція видалення працює правильно і зміни відображаються у базі даних.

Таким чином, усі три тести перевіряють базові операції CRUD (отримання списку, отримання за ID, видалення) на рівні інтеграції. Вони дозволяють впевнитися, що серверна частина застосунку коректно взаємодіє з базою даних, а також відповідає на запити клієнтів згідно з очікуваною логікою.

ВИСНОВКИ

У ході виконання курсової роботи було підтверджено, що автоматична генерація надійних паролів є необхідним інструментом для забезпечення безпеки сучасних інформаційних систем у зв'язку зі зростаючими обсягами електронних даних і постійними загрозами кібербезпеки. Об'єктом дослідження стали процеси генерації та управління паролями, а предметом – алгоритми та методи їх реалізації у клієнт-серверній архітектурі.

Метою роботи було створення програмного забезпечення, яке дозволяє ефективно і зручно генерувати складні паролі, забезпечуючи при цьому належний рівень безпеки. Для досягнення цієї мети були успішно виконані такі завдання: аналіз існуючих підходів, розробка модулів генерації паролів з підтримкою різних алгоритмів, створення веб-інтерфейсу для користувачів, реалізація REST API для інтеграції з іншими сервісами та впровадження Swagger UI для зручної документації і тестування.

Розроблена система забезпечує гнучке і безпечне управління паролями, сприяючи підвищенню рівня захисту даних і зручності користувачів. Запропоноване рішення може бути використане як у персональних, так і корпоративних інформаційних системах, що робить його актуальним і перспективним у контексті сучасних вимог до кібербезпеки.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Bloch J. Effective Java. – 3rd ed. – Boston: Addison-Wesley, 2018. – 416 p.
2. Schildt H. Java: The Complete Reference. – 11th ed. – New York: McGraw-Hill, 2019. – 1248 p.
3. Sierra K., Bates B. Head First Java. – 2nd ed. – Sebastopol: O'Reilly Media, 2005. – 688 p.
4. Eckel B. Thinking in Java. – 4th ed. – Upper Saddle River: Prentice Hall, 2006. – 1150 p.
5. Booch G. Object-Oriented Analysis and Design with Applications. – 3rd ed. – Boston: Addison-Wesley, 2007. – 720 p.
6. Tahchiev P. et al. JUnit in Action. – 2nd ed. – Greenwich: Manning Publications, 2010. – 504 p.
7. Koskela L. Effective Unit Testing. – Shelter Island: Manning Publications, 2013. – 240 p.
8. Oaks S. Java Performance: The Definitive Guide. – Sebastopol: O'Reilly Media, 2014. – 425 p.
9. Naftalin M., Wadler P. Java Generics and Collections. – Sebastopol: O'Reilly Media, 2007. – 286 p.
10. Reese G. Java Database Best Practices. – Sebastopol: O'Reilly Media, 2003. – 294 p.
11. Bauer C., King G. Java Persistence with Hibernate. – 2nd ed. – Greenwich: Manning Publications, 2015. – 608 p.
12. Keith M., Schincariol M. Pro JPA 2: Mastering the Java Persistence API. – 2nd ed. – New York: Apress, 2013. – 636 p.
13. Walls C. Spring in Action. – 5th ed. – Greenwich: Manning Publications, 2018. – 520 p.
14. Cosmina I., Harrop R., Schaefer C., Ho C. Pro Spring 5. – New York: Apress, 2017. – 866 p.
15. Pollack M., Gierke O., Risberg T., Brisbin J., Hunger M. Spring Data. – New

York: Apress, 2013. – 288 p.

16. Carnell J. Spring Microservices in Action. – Greenwich: Manning Publications, 2017. – 384 p.

17. Long J., Bastani K. Cloud Native Java: Designing Resilient Systems with Spring Boot, Spring Cloud, and Cloud Foundry. – Sebastopol: O'Reilly Media, 2017. – 648 p.

18. Spilca L. Spring Security in Action. – Greenwich: Manning Publications, 2020. – 576 p.

19. Rajput D. Spring Boot Testing: Test your Spring Boot applications with JUnit 5. – Birmingham: Packt Publishing, 2020. – 384 p.

ДОДАТОК А

Клас PasswordEntity.java:

```
package com.example.demo.entity;
import java.time.LocalDateTime;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
@Entity
@Table(name = "Passwords")
public class PasswordEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String password;
    private int length;
    @ManyToOne
    @JoinColumn(name = "type_id", nullable = false)
    private PWTypeEntity pwType;
    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private UserEntity user;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public int getLength() {
        return length;
    }
    public void setLength(int length) {
        this.length = length;
    }
    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }
    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }
    public LocalDateTime getCreatedAt() {
        return createdAt;
    }
    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }
    public void setPwType(PWTypeEntity typeEntity) {
        this.pwType = typeEntity;
    }
}
```

```
public void setUser(UserEntity userentity) {
    this.user = userentity;
}
public Long getId() {
    return id;
}
public void setId(Long id) {
    this.id = id;
}
public PWTypeEntity getPwType() {
    return pwType;
}
public UserEntity getUser() {
    return user;
}
}
```

ДОДАТОК Б

Клас PasswordController.java:

```
package com.example.demo.controller;
import com.example.demo.entity.PasswordEntity;
import com.example.demo.service.PasswordService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.time.LocalDateTime;
import java.util.List;

@RestController
@RequestMapping("/api/passwords")
@Tag(name = "Паролі", description = "Керування пароллями користувачів")
public class PasswordController {
    private final PasswordService passwordService;

    public PasswordController(PasswordService passwordService) {
        this.passwordService = passwordService;
    }

    @Operation(summary = "Отримати всі паролі", description = "Повертає список усіх збережених паролів у системі")
    @ApiResponse(responseCode = "200", description = "Список паролів успішно отримано")
    @GetMapping
    public List<PasswordEntity> getAllPasswords() {
        return passwordService.findAll();
    }

    @Operation(summary = "Отримати пароль за ID", description = "Повертає деталі конкретного пароля за вказаним ID")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Пароль знайдено"),
        @ApiResponse(responseCode = "404", description = "Пароль не знайдено")
    })
    @GetMapping("/{id}")
    public ResponseEntity<PasswordEntity> getPasswordById(
        @Parameter(description = "ID пароля") @PathVariable Long id) {
        PasswordEntity password = passwordService.findById(id);
        if (password != null) {
            return ResponseEntity.ok(password);
        } else {
            return ResponseEntity.notFound().build();
        }
    }

    @Operation(summary = "Створити новий пароль", description = "Створює новий запис пароля")
    @ApiResponse(responseCode = "200", description = "Пароль успішно створено")
    @PostMapping
    public PasswordEntity createPassword(
        @RequestBody PasswordEntity passwordEntity) {
        passwordEntity.setCreatedAt(LocalDateTime.now());
        passwordEntity.setUpdatedAt(LocalDateTime.now());
    }
}
```

```

return passwordService.savePassword(passwordEntity);
}
@Operation(summary = "Оновити пароль", description = "Оновлює існуючий
пароль за ID")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Пароль успішно оновлено"),
    @ApiResponse(responseCode = "404", description = "Пароль не знайдено")
})
@PutMapping("/{id}")
public ResponseEntity<PasswordEntity> updatePassword(
    @Parameter(description = "ID пароля, який потрібно оновити") @PathVariable
    Long id,
    @RequestBody PasswordEntity updatedPassword) {
    PasswordEntity existing = passwordService.findById(id);
    if (existing != null) {
        existing.setPassword(updatedPassword.getPassword());
        existing.setLength(updatedPassword.getLength());
        existing.setPwType(updatedPassword.getPwType());
        existing.setUser(updatedPassword.getUser());
        existing.setUpdatedAt(LocalDateTime.now());
        return ResponseEntity.ok(passwordService.savePassword(existing));
    } else {
        return ResponseEntity.notFound().build();
    }
}
@Operation(summary = "Видалити пароль", description = "Видаляє пароль за
ID")
@ApiResponses({
    @ApiResponse(responseCode = "204", description = "Пароль успішно видалено"),
    @ApiResponse(responseCode = "404", description = "Пароль не знайдено")
})
@DeleteMapping("/{id}")
public ResponseEntity<Void> deletePassword(
    @Parameter(description = "ID пароля, який потрібно видалити") @PathVariable
    Long id) {
    PasswordEntity existing = passwordService.findById(id);
    if (existing != null) {
        passwordService.deleteById(id);
        return ResponseEntity.noContent().build();
    } else {
        return ResponseEntity.notFound().build();
    }
}
}
}

```